CSE 2321 · FOUNDATIONS I



Recursion

Turning recursive code into a recurrence, and three ways to solve one.

Greg Ryslik · The Ohio State University

$T(n) = aT(n/b) + f(n)$

Three Lectures, Three Methods

AGENDA

PART I	Recursive algorithms, and writing their recurrences
PART II	Method 1: unrolling a recurrence
PART III	Method 2: recursion trees, then proving the guess
PART IV	Method 3: the master theorem

No single method solves every recurrence. By the end you should know which one to reach for.

Recursion and Recurrences

A problem solved by smaller copies of itself, and the equation that counts its cost.

The Idea: Divide and Conquer

THE IDEA

Solve a big problem by solving **smaller copies of the same problem** , then stitching their answers together.

THINK OF IT AS

Counting the people in a lecture hall: ask each row to count itself, then add up the row totals. Each row can do the same thing seat by seat.

1 Split the problem into smaller instances of the same problem.

DIVIDE

2 Solve each smaller instance the same way. When an instance is small enough, solve it directly instead.

CONQUER · BASE CASE

3 Combine the smaller answers into the answer for the original problem.

COMBINE

4 Without a base case the splitting never stops — the recursive version of an infinite loop.

Merge Sort

THE IDEA

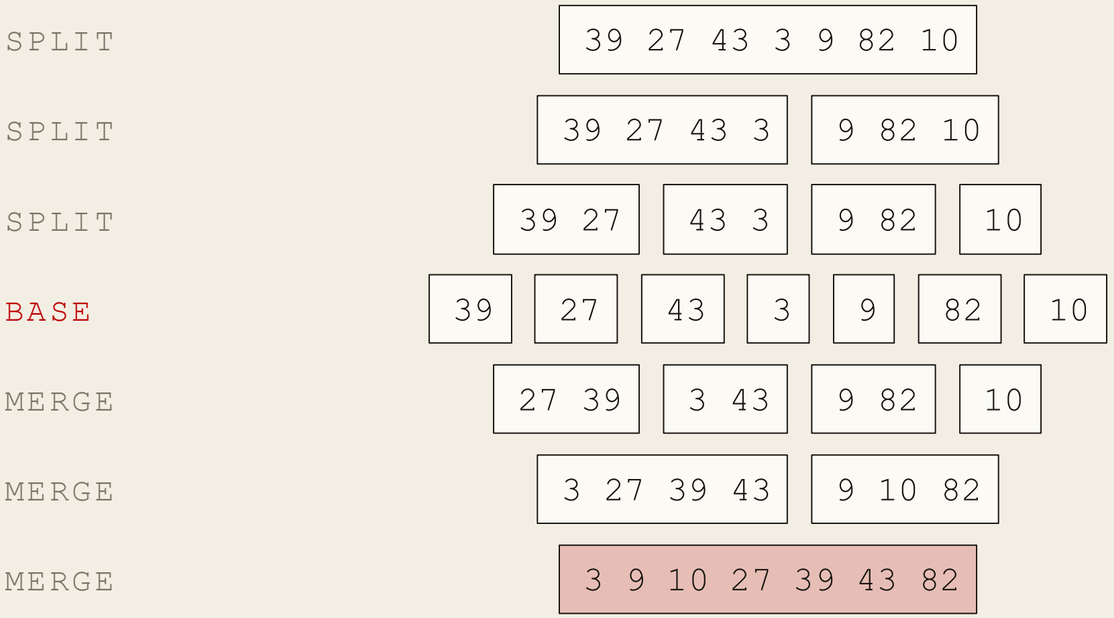
Split the list in half, sort each half, then **merge** the two sorted halves into one.

INPUT REQUIRED

STRUCTURE	Array `A[i..j]`, sorted in place
ORDERING	None required
ELEMENTS	Anything totally ordered

1. If the list has one element, it is already sorted. Stop.
   BASE CASE
2. Split the list into a left half and a right half.
   DIVIDE
3. Sort each half by merge sort.
   CONQUER
4. Merge: repeatedly compare the front of each half and move the smaller one to the output.
   COMBINE

Trace: Merge Sort on 39 27 43 3 9 82 10



Three levels of splitting reach single elements — the base case.

Three levels of merging rebuild the list, sorted.

Seven elements, $\lceil \log_2 7 \rceil = 3$ levels each way.

Trace: The Final Merge

k	front of L	front of R	take	output so far
1	3	9	3	3
2	27	9	9	3 9
3	27	10	10	3 9 10
4	27	82	27	3 9 10 27
5	39	82	39	… 27 39
6	43	82	43	… 39 43
7	∞	82	82	… 43 82

`L = 3 27 39 43 R = 9 10 82`

One comparison per output position, so merging two halves of total size `n` costs `Θ(n)`.

Each half ends with a sentinel ∞. When `L` runs out, its ∞ loses every comparison and the rest of `R` drains out.

Merge Sort, Formally


```
MERGESORT(A[], i, j)
1: if (i < j) then
2:   midp ← ⌊(i + j)/2⌋
3:   MergeSort(A, i, midp)
4:   MergeSort(A, midp + 1, j)
5:   Merge(A, i, midp, j)
6: end if
```

Two calls on half the input, plus a linear merge: $T(n) = 2T(n/2) + Cn$

```
MERGE(A[], first, midp, last)
 1: L ← A[first..midp],  append ∞
 2: R ← A[midp+1..last], append ∞
 3: i ← 1;  j ← 1
 4: for k ← first to last do
 5:   if (L[i] ≤ R[j]) then
 6:     A[k] ← L[i];  i ← i + 1
 7:   else
 8:     A[k] ← R[j];  j ← j + 1
 9:   end if
10: end for
```

The ∞ sentinels are a correction: without them, line 5 reads past the end of whichever half runs out first.

Recurrences

DEFINITION

A **recurrence** is an equation or inequality that describes a function in terms of its values on smaller inputs.

$$T(n) = \begin{cases} C & \text{if } n = 1; \\ 2T(n/2) + Cn & \text{if } n > 1 \end{cases}$$

READING ONE OFF THE CODE

Each recursive call on size `s` contributes `T(s)`.

All other work is counted as usual — loops, comparisons, assignments.

The code's base case becomes the recurrence's base case.

Merge sort's recurrence reads as: two calls of size `n/2`, plus `Cn` to merge.

Reading a Recurrence: factorial(n)

line	cost
`if (n = 0) then`	`C`
`  return 1`	
`else`	
`  return n · factorial(n − 1)`	**`C + T(n − 1)`**

$$T(n) = T(n - 1) + C,\ T(0) = C$$

TRACE: factorial(3)

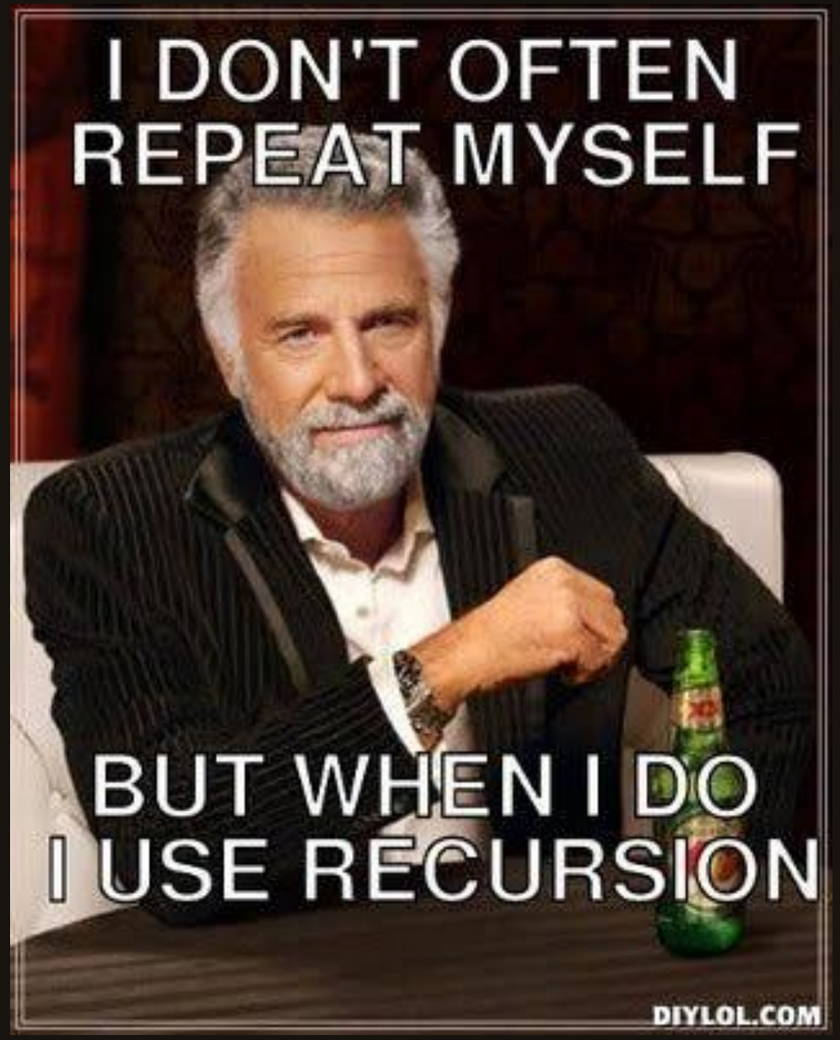
```
factorial(3) → 3 · factorial(2)
factorial(2) → 2 · factorial(1)
factorial(1) → 1 · factorial(0)
factorial(0) → 1
returns: 1, 1, 2, 6
```

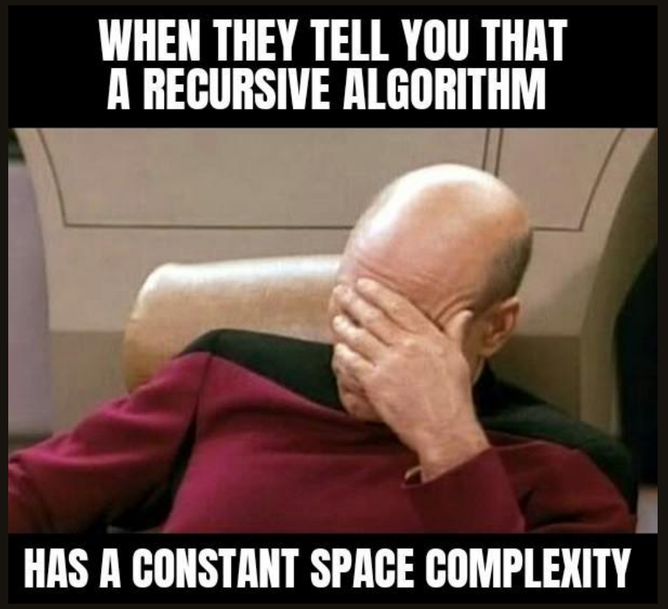
Four calls for `n = 3`: one per value from `3` down to `0`.

BASE CASE NOT FOUND

To understand recursion, you must first understand recursion.

The joke works because it has no base case. Neither does a stack overflow.





Method 1: Unrolling

Substitute the recurrence into itself until a pattern appears.

The Idea: Unroll It

THE IDEA

Replace the smaller term by its own recurrence, again and again, until you can see what **k** rounds of replacement look like.

THINK OF IT AS

Unpacking a set of nesting dolls and counting what you pull out at each layer, until you reach the solid one in the middle.

1. Write the recurrence for the smaller input, and substitute it in.
   EXPAND
2. Repeat two or three times, and keep the work you have collected.
   EXPAND AGAIN
3. Write the general form after k rounds.
   SPOT THE PATTERN
4. Choose k so the remaining T(·) is the base case, then simplify the sum.
   HIT THE BASE

Unrolling: T(n) = T(n − 1) + C

	$T(n) = T(n-1) + C$	the recurrence
	$= T(n-2) + 2C$	$T(n-1) = T(n-2) + C$
	$= T(n-3) + 3C$	$T(n-2) = T(n-3) + C$
k	$= T(n-k) + kC$	after k rounds
BASE	$= T(0) + nC$	choose k = n
	$= (n+1)C$	$T(0) = C$

$T(n) \in \Theta(n)$ For $n = 3$ that is $4C$ — one C per call in the trace.

Selection Sort, Recursively

THE IDEA

Push the largest element to the end, then sort **everything before it** the same way.

INPUT REQUIRED

STRUCTURE	Array `A[1..n]`, sorted in place
ORDERING	None required
ELEMENTS	Anything totally ordered

1. If there is at most one element, stop.
   BASE CASE
2. Compare each earlier element with the last one, and swap whenever the earlier one is bigger.
   ONE PASS
3. The largest value is now in the last slot, and it is finished.
4. Sort the first n − 1 elements the same way.
   RECURSE

Selection Sort: Trace and Code

n	pass compares	array after pass
4	4>7, 9>7 swap, 2>9	4 7 2 **9**
3	4>2 swap, 7>4 swap	2 4 **7** 9
2	2>4	2 **4** 7 9
1	base case	2 4 7 9

3 + 2 + 1 = 6 comparisons for n = 4.

SELECTIONSORT(A[], n)

```
1: if (n ≤ 1) then return
2: for i ← 1 to n − 1 do
3:   if (A[i] > A[n]) then
4:     swap(A[i], A[n])
5:   end if
6: end for
7: SelectionSort(A, n − 1)
```

$T(n) = T(n - 1) + Cn$

Unrolling: T(n) = T(n − 1) + Cn

	$T(n) = T(n-1) + Cn$	the recurrence
	$= T(n-2) + C(n-1) + Cn$	expand once
	$= T(n-3) + C(n-2) + C(n-1) + Cn$	expand again
BASE	$= T(0) + C(1 + 2 + \cdots + n)$	choose k = n
	$= C + C \cdot n(n+1)/2$	sum formula, proved by induction

$T(n) \in \Theta(n^2)$

The work per round shrinks by one each time, and the rounds add up to the same arithmetic sum as bubble sort's comparisons.

Binary Search, Recursively

BINSEARCHREC(A[], i, j, K)

```
1: if (i ≤ j) then
2:   midp ← ⌊(i + j)/2⌋
3:   if (K = A[midp]) return midp
4:   else if (K < A[midp]) then
5:     return BinSearchRec(A, i, midp − 1, K)
6:   else
7:     return BinSearchRec(A, midp + 1, j, K)
8: else return 0
```

$T(n) = T(n/2) + C$

INPUT REQUIRED

STRUCTURE	Array with direct index access
ORDERING	**Sorted increasing**
RETURNS	Position, or 0

call	i	j	midp	A[midp]
1	1	8	4	11
2	5	8	6	19 ✓

A = 2 5 7 11 14 19 23 30, K = 19.

Unrolling: T(n) = T(n/2) + C

	$T(n) = T(n/2) + C$	the recurrence
	$= T(n/4) + 2C$	T(n/2) = T(n/4) + C
	$= T(n/8) + 3C$	expand again
k	$= T(n/2^k) + kC$	after k rounds
BASE	$n/2^k = 1 \Longrightarrow k = \log_2 n$	size reaches 1
	$T(n) = C + C \log_2 n$	T(1) = C

$T(n) \in \Theta(\log n)$ Take n a power of 2. Floors change nothing asymptotically.

Unrolling: Recursive Fibonacci

`fib(n)` calls `fib(n − 1)` and `fib(n − 2)`: `T(n) = T(n − 1) + T(n − 2) + C`. Since `T(n − 1) ≥ T(n − 2)`, we get `T(n) ≥ 2T(n − 2)`.

	`T(n) ≥ 2T(n − 2)`	drop C, use T(n−1) ≥ T(n−2)
	`≥ 2² T(n − 4)`	expand
`k`	`≥ 2ᵏ T(n − 2k)`	after k rounds
`BASE`	`≥ 2^(n/2) · C`	choose k = n/2

CORRECTION

The source concludes `Ω(2ⁿ)`. What was shown is

`T(n) ∈ Ω(2^(n/2)) = Ω(1.414ⁿ)`

and `2ⁿ` grows strictly faster. The true rate is `Θ(φⁿ)`, `φ ≈ 1.618`.

When the Input Shrinks by a Fraction

line	cost
`if (n ≤ 20) return A[n]`	C
`for i ← 1 to ⌊n/2⌋ do`	Cn
`  A[i] ← A[i] + A[2i]`	
`x ← func(A, ⌊3n/5⌋)`	**T(3n/5)**

$$T(n) = T(3n/5) + Cn$$

Each call does linear work, then recurses on 60% of its input.

Will the shrinking sizes add up to something bigger than n ? Unroll and see.

Unrolling: T(n) = T(3n/5) + Cn

	$T(n) = T(3n/5) + Cn$	the recurrence
	$= T(9n/25) + (3/5)Cn + Cn$	expand once
k	$= T((3/5)^k n) + Cn\ [1 + 3/5 + \cdots + (3/5)^{k-1}]$	after k rounds
UPPER	$< C + Cn \cdot 1/(1 - 3/5) = C + (5/2)Cn$	infinite geometric sum
LOWER	$> Cn$	the first term alone

$T(n) \in \Theta(n)$ When the sizes shrink geometrically, the **first** call's work dominates the total.

A Loop Inside the Recursion

FUNC(A[], n)

```
1: if (n ≤ 10) then return A[n]
2: for i ← 1 to ⌊n/2⌋ do
3:   j ← 7
4:   while (j ≤ ⌊n/2⌋) do
5:     A[i] ← A[i] + A[i + j]
6:     j ← 5 · j
7:   end while
8: end for
9: x ← func(A, n − 8)
```

COUNT THE WHILE LOOP

$j = 7, 35, 175, \ldots, 7 \cdot 5^i$

runs while $7 \cdot 5^i \le n/2$

$\approx \log_5(n/14) \in \Theta(\log n)$ times

Outer loop $n/2$ times, inner $\Theta(\log n)$ each:

$$T(n) = T(n - 8) + Cn \log n$$

Unrolling: T(n) = T(n − 8) + Cn log n

After $k = n/8$ rounds: $T(n) = C + C\,\Sigma(n - 8i)\,\log(n - 8i)$, the sum over $i = 0, \ldots, n/8 - 1$.

UPPER: EVERY TERM ≤ n log n

$$\leq C + (n/8) \cdot Cn \log n$$

$$= C + (C/8)\, n^2 \log n$$

LOWER: KEEP THE BIG HALF

$n/16$ terms, each $\geq (n/2) \log(n/2)$

$$\geq (C/64)\, n^2 \log n \quad n \geq 4$$

$$T(n) \in \Theta(n^2 \log n)$$

The lower bound uses $\log(n/2) \geq \frac{1}{2} \log n$, which holds for $n \geq 4$.

Extra Practice

(a) PROVE BY INDUCTION

$T(n) = T(3n/5) + Cn,\ T(1) = C$

Show $T(n) \leq (5/2)Cn$ for all $n \geq 1$.

(b) UNROLL

$T(n) = 2T(n - 1) + C,\ T(0) = C$

Find a closed form, and its Θ.

For (a), assume the bound for all smaller inputs — strong induction. For (b), the sum you collect is geometric.

Extra Practice: Answers

(a)

BASE	$T(1) = C \le (5/2)C$	✓
HYP	$T(m) \le (5/2)Cm$, all $m < n$	strong
STEP	$T(n) \le (5/2)C(3n/5) + Cn$	hypothesis
	$= (3/2)Cn + Cn = (5/2)Cn$	✓

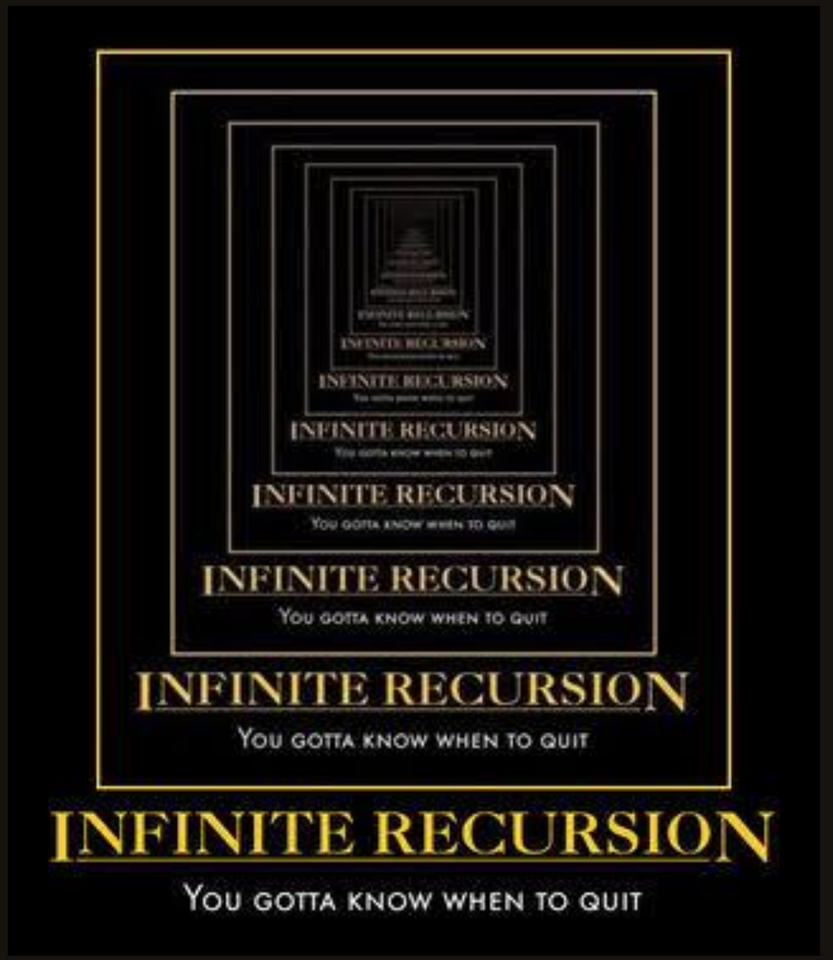
(b)

	$T(n) = 2T(n-1) + C$	
k	$= 2^k T(n-k) + C(1 + 2 + \cdots + 2^{k-1})$	
BASE	$= 2^n C + C(2^n - 1)$	k = n
	$= C(2^{n+1} - 1) \in \Theta(2^n)$	✓

Check (b) at $n = 1$: $T(1) = 2C + C = 3C$, and $C(2^2 - 1) = 3C$ ✓.

Method 2: Recursion Trees

Draw every call, total each level, then prove the guess by induction.



The Idea: Draw the Calls

THE IDEA

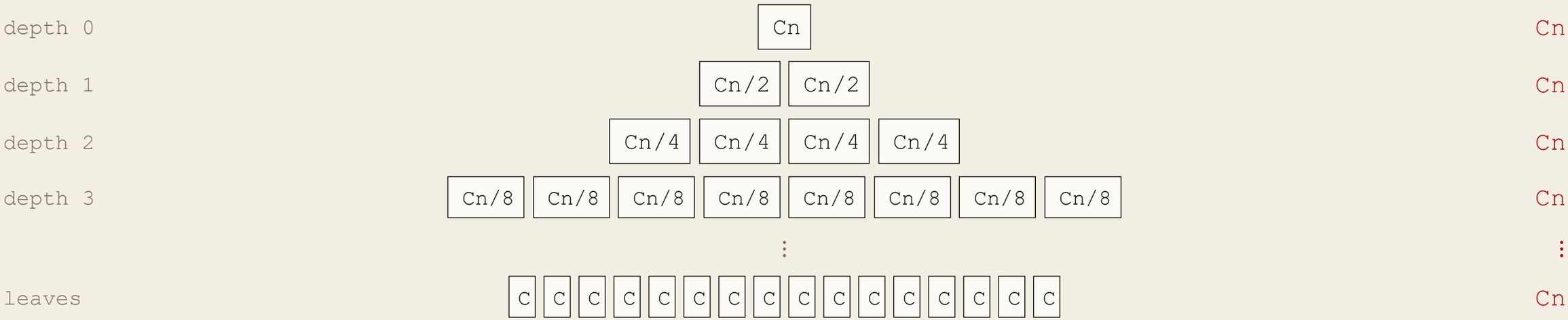
Draw one node per call, labelled with the work it does **outside** its recursive calls. The total cost is the sum of every node.

THINK OF IT AS

An org chart where every box lists the hours that person works themselves. Total hours for the company is every box added up — easiest done one management level at a time.

1. Root: the work the first call does, ignoring its subcalls.
   ROOT
2. Children: one per recursive call, labelled the same way at the smaller size.
   EXPAND
3. Add up each level of the tree separately.
   LEVEL TOTALS
4. Find the number of levels, then add the level totals.
   SUM

Tree for T(n) = 2T(n/2) + Cn



$Cn \cdot (\log_2 n + 1) \in \Theta(n \log n)$

Every level totals Cn: twice as many nodes, each half the size. n leaves at depth $\log_2 n$.

Tree for $T(n) = 3T(n/4) + cn^2$

depth	nodes	size	cost per node	level total
0	1	n	cn^2	cn^2
1	3	$n/4$	$c(n/4)^2$	$(3/16)cn^2$
2	9	$n/16$	$c(n/16)^2$	$(3/16)^2cn^2$
i	3^i	$n/4^i$	$c(n/4^i)^2$	$(3/16)^icn^2$
leaves	$3^{(\log_4 n)} = n^{(\log_4 3)}$	1	$T(1)$	$\Theta(n^{0.79})$

$$T(n) < cn^2 \cdot \Sigma(3/16)^i + \Theta(n^{0.79})$$

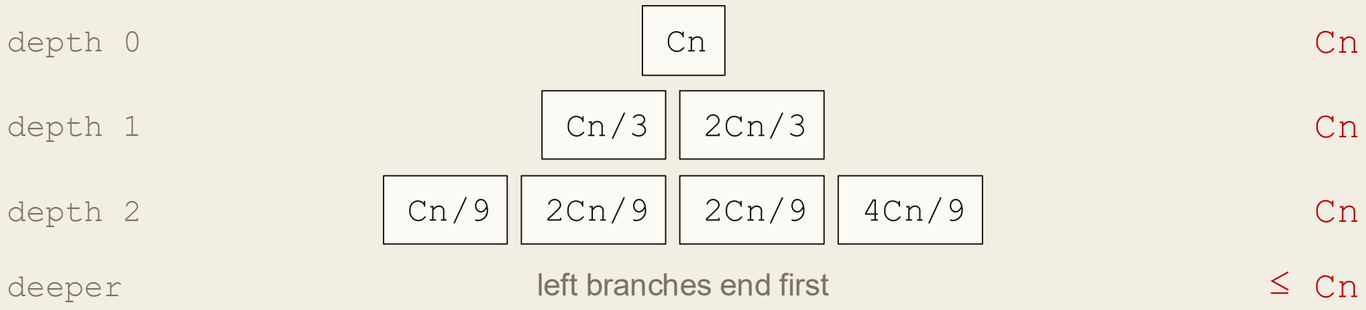
$$= (16/13)cn^2 + \Theta(n^{0.79})$$

geometric sum

$\Theta(n^2)$

Lower bound: the root alone costs cn^2.

Tree for T(n) = T(n/3) + T(2n/3) + Cn



Shortest path $n \to n/3 \to \cdots$ has $\log_3 n$ levels, all complete.

Longest path $n \to 2n/3 \to \cdots$ has $\log_{3/2} n$ levels.

BOTH BOUNDS

$$Cn \log_3 n \le T(n) \le Cn (\log_{3/2} n + 1)$$

$\Theta(n \log n)$

The source wrote this as an equality. Lower levels are incomplete, so it is only an upper bound.

Guess, Then Prove It

THE SUBSTITUTION METHOD

1. Guess the form of the answer — from a tree, or from unrolling.
2. Prove it by induction, finding the constants as you go.

For an upper bound $T(n) \in O(g(n))$: assume $T(m) \leq c \cdot g(m)$ for every smaller m, substitute into the recurrence, and show $T(n) \leq c \cdot g(n)$ — with the **same** c.

This is strong induction. The hypothesis has to cover whatever smaller sizes the recurrence calls.

Proof: T(n) = 2T(⌊n/2⌋) + n is O(n log n)

$T(1) = 1$. Claim: $T(n) \le 2n \log n$ for all $n \ge 2$.

BASE	$T(2) = 2T(1) + 2 = 4 \le 2 \cdot 2 \cdot \log 2 = 4$	n = 2 ✓
BASE	$T(3) = 2T(1) + 3 = 5 \le 2 \cdot 3 \cdot \log 3 \approx 9.5$	n = 3 ✓
HYP	$T(m) \le 2m \log m$ for $2 \le m < n$	strong, arbitrary n ≥ 4
STEP	$T(n) \le 2 \cdot 2\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n$	hypothesis: ⌊n/2⌋ ≥ 2
	$\le 2n \log(n/2) + n$	⌊n/2⌋ ≤ n/2
	$= 2n \log n - 2n + n$	log(n/2) = log n − 1
	$\le 2n \log n$	✓

Which Base Cases?

$n = 1$ CANNOT BE A BASE CASE

$T(1) = 1$

$c \cdot 1 \cdot \log 1 = 0$

$1 \leq 0$ is false, for every c

SO 2 AND 3 MUST BE

$n = 2, 3$: $\lfloor n/2 \rfloor = 1$ — outside the claim

$n \geq 4$: $\lfloor n/2 \rfloor \geq 2$ — covered

The base case of the **recurrence** and the base cases of the **proof** are different things.

Big O only needs the bound for $n \geq n_0$, so starting at $n_0 = 2$ is allowed. Check every size whose recursive call falls outside the claim.

Two Traps

A LOOSE BOUND IS STILL TRUE

If $T(n) \in \Theta(n)$, you can still prove $T(n) \leq cn^2$.

True, and useless. Substitution checks a guess; it does not tell you the guess is tight.

A FAKE PROOF THAT T(n) = O(n)

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

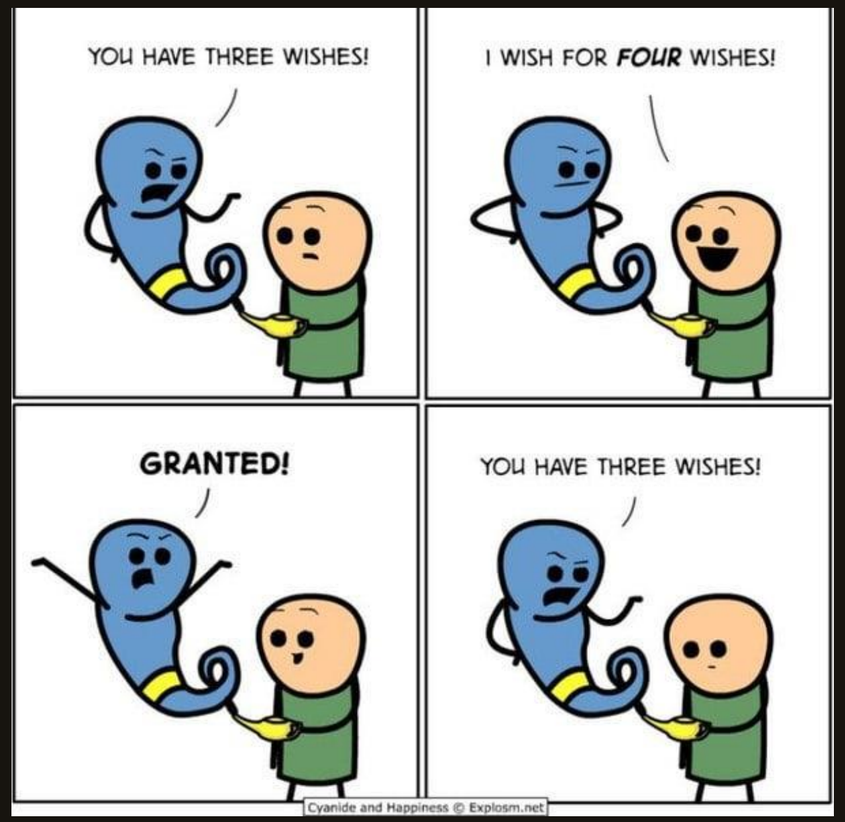
$$\leq 2c\lfloor n/2 \rfloor + n$$

$$\leq cn + n$$

"$= O(n)$" ✗

The hypothesis was $T \leq cn$, so the step must end at $\leq cn$ — the **same** c. Ending at $(c + 1)n$ proves nothing: over $\log n$ levels those extra n's add up to $n \log n$.

Method 3: The Master Theorem



Compare the work at the root with the work at the leaves.

The Idea: Root Versus Leaves

THE IDEA

In the tree for `aT(n/b) + f(n)`, compare the root's work `f(n)` with the number of leaves, `n^(log_b a)`. **Whichever is bigger wins.**

THINK OF IT AS

Three trees from Part III: 3T(n/4) + cn² was top-heavy, 2T(n/2) + Cn was even at every level. Case 1 is the opposite of the first.

1 Leaves are much heavier: the bottom of the tree dominates.

`CASE 1 · Θ(n^(log_b a))`

2 Root and leaves balance: every level costs about the same, and there are log n levels.

`CASE 2 · Θ(n^(log_b a) log n)`

3 Root is much heavier: the top of the tree dominates.

`CASE 3 · Θ(f(n))`

Where $n^{\log_b a}$ Comes From

Each call splits into a calls of size n/b. The tree has depth $\log_b n$, so it has

$$a^{\log_b n} = n^{\log_b a}$$

leaves, each doing constant work.

$a^{\log_b n}$	leaves
$= (b^{\log_b a})^{\log_b n}$	$a = b^{\log_b a}$
$= (b^{\log_b n})^{\log_b a}$	swap exponents
$= n^{\log_b a}$	$b^{\log_b n} = n$

recurrence	a	b	$n^{\log_b a}$	leaves
$2T(n/2) + Cn$	2	2	n^1	n
$3T(n/4) + cn^2$	3	4	$n^{0.79}$	fewer than n
$9T(n/3) + n$	9	3	n^2	n^2

The Master Theorem

Let $a \geq 1, b > 1, f(n) \geq 0$, and $T(n) = aT(n/b) + f(n), T(1) = \Theta(1)$.

CASE 1	$f(n) \in O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$	$T(n) \in \Theta(n^{\log_b a})$	f is polynomially smaller
CASE 2	$f(n) \in \Theta(n^{\log_b a})$	$T(n) \in \Theta(n^{\log_b a} \log n)$	same size
CASE 3	$f(n) \in \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and $af(n/b) \leq cf(n)$ for some $c < 1$, all large n	$T(n) \in \Theta(f(n))$	f is polynomially larger, and regular

"Polynomially" means by a factor of n^{ε}. A factor of $\log n$ is not enough. Replacing n/b by $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$ does not change the answer.

Applying It: Four Steps

01 Check the form `aT(n/b) + f(n)`: equal-size pieces, divided — not subtracted.

02 Read off `a`, `b`, and `f(n)`. Compute `n^(log_b a)`.

03 Compare `f(n)` with `n^(log_b a)`: polynomially smaller, the same, or polynomially larger.

04 In case 3, check regularity: `af(n/b) ≤ cf(n)` for some `c < 1`.

If none of the three cases fits, the theorem says nothing. Use a tree or unroll instead.

Five Recurrences, One Comparison

recurrence	a, b	n^(log_b a)	f(n)	case	T(n)
9T(n/3) + n	9, 3	n^2	n	1	Θ(n^2)
2T(n/2) + Cn merge sort	2, 2	n	n	2	Θ(n log n)
T(n/2) + C binary search	1, 2	1	1	2	Θ(log n)
T(2n/3) + 1	1, 3/2	1	1	2	Θ(log n)
3T(n/4) + n log n	3, 4	n^0.79	n log n	3	Θ(n log n)

Case 1: $n = O(n^{(2-1)}), \varepsilon = 1$. Case 3: $n \log n = \Omega(n^{(0.79 + 0.2)}), \varepsilon = 0.2$ — regularity on the next slide.

Case 3 Needs a Regularity Check

For $T(n) = 3T(n/4) + n \log n$, show $af(n/b) \leq cf(n)$ with $c < 1$:

$a\, f(n/b) = 3 \cdot (n/4) \log(n/4)$	substitute
$= (3/4)\, n \log(n/4)$	simplify
$\leq (3/4)\, n \log n$	$\log(n/4) \leq \log n$

$c = 3/4 < 1$ ✓ Regularity says each level costs at most a fixed fraction of the level above it, so the root really does dominate.

When the Master Theorem Says Nothing

LARGER, BUT NOT POLYNOMIALLY

$T(n) = 2T(n/2) + n \log n$

$n^{(\log_b a)} = n$, and $f(n)/n = \log n$ — smaller than every n^{ε}. No case fits.

A tree still solves it: level i costs $n(\log n - i)$, totalling

$$n \log n \, (\log n + 1)/2 \in \Theta(n \log^2 n)$$

NOT THE RIGHT SHAPE

$T(n - 1) + Cn$ — subtracts instead of dividing. Unroll.

$T(n/3) + T(2n/3) + Cn$ — unequal pieces. Draw the tree.

$T(n - 1) + T(n - 2) + C$ — Fibonacci. Unroll for a bound.

Reference: aT(n/b) + f(n)

recurrence	n^(log_b a)	case	T(n)
T(n/2) + c, T(n/3) + c	1	2	Θ(log n)
T(n/2) + cn, T(n/3) + cn	1	3	Θ(n)
2T(n/2) + cn, 3T(n/3) + cn	n	2	Θ(n log n)
3T(n/2) + cn	n^1.585	1	$\Theta(n^{(\log_2 3)})$
4T(n/2) + cn	n^2	1	$\Theta(n^2)$
2T(n/2) + cn^2	n	3	$\Theta(n^2)$
4T(n/2) + cn^2	n^2	2	$\Theta(n^2 \log n)$

Highlighted row: the source wrote $\Theta(n^{(\log_4 3)})$. The exponent is $\log_2 4 = 2$.

Reference: Unequal and Subtract-One

$T(\alpha_1 n) + \cdots + T(\alpha_k n) + cn$

$\alpha_1 + \cdots + \alpha_k = 1 \rightarrow \Theta(n \log n)$

$T(n/3) + T(2n/3), T(n/4) + T(3n/4), T(n/6) + T(2n/6) + T(3n/6)$

$\alpha_1 + \cdots + \alpha_k < 1 \rightarrow \Theta(n)$

$T(n/4) + T(n/2), T(n/5) + T(3n/5), T(n/6) + T(4n/6)$

SUBTRACT-ONE, $f(n) \geq 0$

$aT(n-1) + f(n) \rightarrow \Omega(a^n)$

e.g. $2T(n-1) + f(n) \in \Omega(2^n)$

$T(n-1) + T(n-2) + f(n) \rightarrow \Omega(2^{(n/2)})$

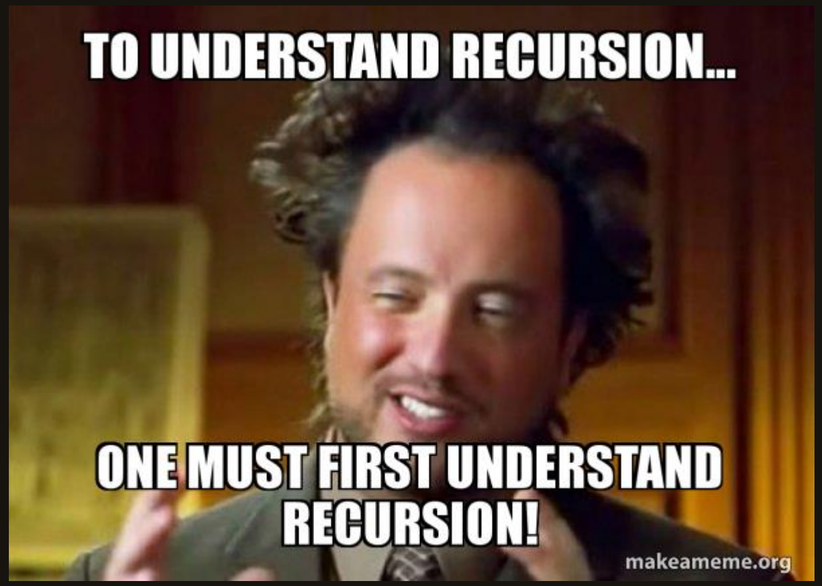
true but loose — the actual rate is φ^n

Fractions summing to 1 means every complete level of the tree costs exactly cn . Summing to less than 1 makes the levels shrink geometrically, like $T(3n/5) + Cn$.

```
a = 2, b = 2, f(n) = n log n
```

Plugging into the master theorem, feeling unstoppable.

Then f(n) turns out to be bigger by a log factor, and none of the three cases applies.



Common Mistakes

BEFORE YOU GO

01 Ending a substitution proof at `(c + 1)n` and calling it `O(n)`. The constant must come out exactly as it went in.

02 Using the recurrence's base case as the proof's base case. `T(1) ≤ c · 1 · log 1 = 0` is false.

03 Applying the master theorem to `T(n − 1)` or to unequal pieces. It needs `aT(n/b)`.

04 Treating a `log n` gap as polynomial. `2T(n/2) + n log n` is not case 3.

05 Reading `Ω(2^(n/2))` as `Ω(2ⁿ)`. Exponents inside an exponential matter.

Recap

01 A recursive algorithm’s cost is a recurrence: each call becomes `T(size)`, all other work counted as usual.

02 **Unrolling**: substitute until a pattern appears, then choose `k` to hit the base case.

03 **Recursion trees**: total each level, then add the levels. Use them to guess.

04 **Substitution**: prove the guess by induction, with the same constant on both sides.

05 **Master theorem**: compare `f(n)` with $n^{\log_b a}$ — leaves win, tie, or root wins.

Merge sort is Θ(n log n) by every one of these methods. Now you can show it three ways.